

2026量子计算黑客松决赛论文

胡书鸣 中国科学技术大学

指导老师：陈凯教授

摘要

本文面向量子多目标组合优化任务，研究有限量子采样预算下的 Pareto 前沿搜索以及大规模候选样本的经典后处理问题。针对多目标 QAOA 求解过程中权重空间覆盖与单方向采样深度之间的矛盾，首先采用 Simplex Lattice 对五目标权重空间进行规则离散，以 $H = 6$ 构造 210 个权重方向，并通过第一阶段均匀采样获得广覆盖候选解和方向级反馈；随后以各方向的非支配解发现能力作为评价指标，筛选高价值权重进入第二阶段，并按照方向排名采用指数衰减方式分配剩余 shots，从而将有限采样资源集中于更具潜力的搜索方向。针对大规模经典后处理任务，结合规则二维格点结构优化多目标 Ising 能量计算，并通过分块处理、轻量预筛选、局部非支配筛选和双前沿交叉合并实现增量式 Pareto 前沿维护。实验结果表明，上述方法能够有效提高有限 shots 下的搜索质量，减少目标值计算与非支配判断过程中的无效开销。

1 问题背景与描述

1.1 变分量子算法与 QAOA

近年来，量子计算技术不断发展，量子算法在组合优化、量子化学和机器学习等领域展现出新的研究潜力。然而，现阶段量子设备的量子比特规模、相干时间和门操作精度仍然受到限制，难以直接运行需要深层量子线路的大规模量子算法。在这一背景下，结合参数化量子线路与经典优化器的变分量子算法逐渐成为当前量子计算研究的重要方向^[1]。

变分量子算法通常采用量子与经典协同计算的方式。量子计算部分负责制备参数化量子态并完成测量，经典计算部分根据测量结果更新线路参数，两者通过迭代共同完成优化过程。与深层量子线路相比，这类方法具有线路结构相对灵活、能够适应现阶段量子硬件条件等特点，因此被广泛用于研究近期量子设备上的实际计算问题。

量子近似优化算法（Quantum Approximate Optimization Algorithm, QAOA）是变分量子算法在组合优化领域的重要代表，其原理如图1-1所示。该算法将待求解的组合

优化问题编码为问题哈密顿量，通过问题哈密顿量与混合哈密顿量的交替演化构造参数化量子态，并从测量结果中获得候选解。围绕 QAOA 的线路深度、参数优化、参数迁移和采样效率等问题，已经形成了较为丰富的研究方向^[2-3]。

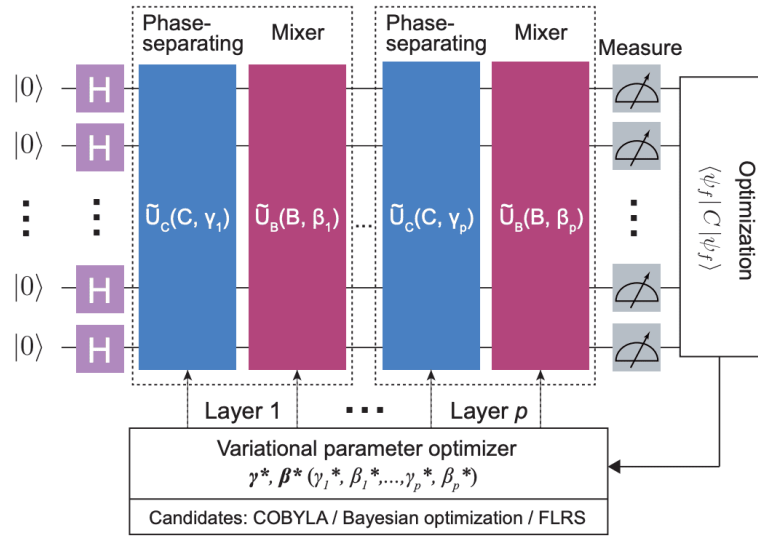


图 1-1 QAOA 基本流程^[4]

传统 QAOA 研究大多以单一目标函数为主要优化对象，而许多实际决策问题往往需要同时考虑多个相互冲突的目标。在这类问题中，算法不仅需要寻找高质量候选解，还需要获得能够反映不同目标权衡关系的解集。如何将 QAOA 的量子采样能力与多目标优化中的权重设计、Pareto 前沿搜索和有限资源分配结合起来，由此成为值得研究的问题。本文所研究的多目标 Ising 优化任务正是在这一背景下展开。

1.2 多目标组合优化问题

多目标组合优化同时面临离散搜索空间庞大和目标之间相互冲突两方面困难。一个候选解可能在某一目标上表现突出，却在其他目标上付出明显代价，因此问题通常不存在同时使所有目标达到最优的唯一解。求解目标也由寻找单个最优点转变为获得一组具有不同权衡关系的非支配解（ND），并尽可能兼顾前沿的收敛程度与覆盖范围^[5-6]。

本赛题以二进制决策变量和多目标 Ising 模型为基础，搜索空间规模随变量数量快速增长。多个目标共享同一组自旋变量和图结构，但局部场与耦合参数不同，各目标的低能区域往往并不重合。由此，权重设计决定搜索偏好，量子采样决定候选解的获取方式，经典后处理则负责从大量样本中提取有效前沿并指导下一轮的搜索方向与采样策略，三者共同影响最终结果。

这类问题广泛存在于物流调度、金融投资、资源配置和通信网络等场景。实际决策往往需要在成本、收益、风险和资源消耗等指标之间进行权衡，因此高质量解集不仅要接近真实 Pareto 前沿，还应覆盖具有代表性的不同偏好区域，如图1-2所示。

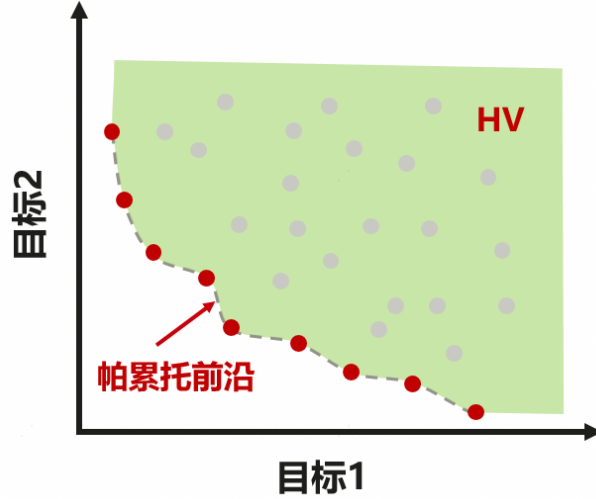


图 1-2 多目标优化中的 Pareto 前沿示意图

本赛题采用最小化形式描述多目标优化问题。设 $\mathbf{z}$ 为离散决策变量， $f(\mathbf{z})$ 为由多个目标组成的目标向量。若一个解在所有目标上均不劣于另一个解，并且至少在一个目标上更优，则称前者支配后者；不被其他可行解支配的解称为 Pareto 最优解，其在目标空间中的映射构成 Pareto 前沿。

1.3 多目标 Ising 优化模型

本赛题将多目标组合优化建模为定义在格点图上的多目标 Ising 能量最小化问题。每个顶点对应一个取值为 -1 或 $+1$ 的自旋变量，多个目标共享同一组自旋变量和图结构，但具有不同的局部场和边耦合参数。第 t 个目标可写为：

$$f_t(\mathbf{z}) = \sum_i h_i^{(t)} z_i + \sum_{(i,j) \in E} J_{ij}^{(t)} z_i z_j$$

小规模任务采用 4×5 格点结构，共 20 个自旋变量和 5 个目标。由于不同目标的低能区域通常不一致，算法需要寻找一组在多个目标之间取得不同平衡的解，而不是只优化某一个 Ising 目标^[7-8]。

赛题使用 Hypervolume (HV) 评价前沿质量。小规模问题会先利用每个目标的精确上下界进行归一化，再以 1.01 为参考点计算 HV。HV 越大，说明当前非支配解集合在目标空间中覆盖的有效区域越大。

1.4 QAOA 与赛题任务

QAOA 通过交替施加问题哈密顿量和混合哈密顿量构造参数化量子态，并通过测量获得候选比特串。面对多目标问题，通常先按照给定权重将多个目标组合为标量子问题，再针对不同权重方向构造相应的问题哈密顿量。权重变化会改变目标偏好，也会影响量子态演化过程及最终测量分布。

本赛题对量子采样总预算作出严格限制，因而无法对大量权重方向同时进行高强度重复采样。**main1** 的关键在于如何选择权重、如何在方向之间分配 **shots**，以及如何利用已经获得的采样结果指导后续投入。**main2** 则需要处理大规模候选样本，其效率主要受到目标值计算和非支配筛选规模的影响。

从任务流程看，**main1** 与 **main2** 分别对应解集质量和经典处理效率两个方向。前者要求在固定 100000 **shots** 预算下输出量子采样结果，并以相对于 **baseline** 的 HV 提升作为主要评价依据；后者要求在保持 HV、前沿目标值和非支配点数量正确的前提下尽可能缩短经典后处理时间。

综合来看，本赛题的求解效果不仅取决于 QAOA 对单个标量子问题的采样能力，还受到权重方向选择、有限采样资源分配以及大规模样本后处理效率等多方面因素的共同影响。针对这些问题，本文从量子采样与经典后处理两个层面展开研究：在量子采样阶段，重点探索如何利用有限的采样预算获得覆盖范围更广、质量更高的候选解集；在经典处理阶段，则结合赛题实例结构和 Pareto 前沿筛选过程，减少目标值计算与非支配判断中的无效开销。在此基础上，本文逐步形成了一套面向多目标 Ising 优化问题的量子与经典协同求解方案，后续章节将对其设计思路、具体实现和实验结果进行详细说明。

2 问题分析

2.1 main1 问题特点与求解难点分析

结合第一章对赛题任务的介绍可以看出，本赛题中 **main1** 问题的困难并不仅来自组合优化问题本身较大的搜索空间。对于多目标 Ising 优化任务，QAOA 需要在不同目标偏好下生成候选解，而最终评价关注的是由这些候选解共同形成的 Pareto 前沿。因此，算法性能不仅取决于单次量子采样能否获得低能量解，还受到权重空间覆盖、采样预算分配等因素的共同影响。

首先，多目标问题需要通过不同权重描述目标之间的偏好关系，而连续权重空间只能通过有限数量的方向进行离散近似。如果权重数量过少或分布不合理，部分目标权衡区域可能难以得到充分探索；如果使用大量权重，又会在总采样预算固定的条件下压缩单个方向能够获得的 shots。由此，权重空间覆盖与单方向采样深度之间形成了直接矛盾。

此外，不同权重方向的实际搜索价值并不完全相同。即使采用较均匀的权重空间离散方式，不同方向产生高质量非支配候选解的能力仍可能存在明显差异。如果始终对所有方向平均分配采样预算，就难以充分利用这些差异；但在采样开始之前，又缺少足够信息判断哪些方向更值得投入。因此，如何从已有采样结果中提取有效反馈，并据此调整后续资源分配，是提高有限 shots 利用效率的另一个关键问题。

综合上述分析，本赛题的优化可以归纳为两个相互联系的核心问题：一是在严格 shots 约束下，如何兼顾权重空间覆盖与重点方向的采样深度；二是如何根据之前的采样结果辅助下一轮方案的设计。

2.2 权重空间与标量化机制

为了利用已有的单目标优化方法求解多目标问题，相关研究通常采用标量化方法，将多个目标按照一定规则组合为标量子问题，并通过求解一系列具有不同目标偏好的子问题获得 Pareto 前沿的近似解集。常见的标量化方法包括线性加权法、切比雪夫方法以及边界交叉类方法等，其中线性加权法具有形式简单、计算开销较低且易于与现有优化框架结合等特点^[5-6,9-10]。

在线性加权方法中，不同权重向量对应不同的目标偏好方向。通过改变权重，可以构造一系列标量子问题，并从不同方向搜索具有代表性的候选解。在多目标 QAOA 研究中，这种处理方式能够较自然地与 Ising 模型结合：不同目标对应的模型系数按照权重进行组合，从而形成不同的加权问题哈密顿量，再利用 QAOA 对相应子问题进行采样^[11]。因此，权重集合的设计会直接影响算法能够探索的目标权衡区域。

对于具有多个目标的问题，满足非负性与归一化约束的权重向量构成一个单纯形。理论上，连续变化的权重可以表示丰富的目标偏好关系，但实际计算只能选取有限数量的权重方向。相关研究通常采用随机采样、均匀采样或基于单纯形格点的系统化方法构造有限权重集合。其中，Simplex Lattice 方法通过给定离散参数，将单纯形

划分为规则分布的权重方向，能够同时覆盖极端方向、边界区域和内部混合区域，因此常被用于分解型多目标优化算法中的权重向量生成^[9]。

与随机生成权重相比，规则的单纯形离散能够减少有限样本条件下的局部聚集现象，也便于控制权重数量和空间分辨率。不过，权重空间的离散粒度与计算资源之间仍然存在明显矛盾。较少的权重方向可能遗漏部分目标权衡区域，而增加权重数量又意味着有限计算预算需要分散到更多子问题中。在量子采样场景下，这一问题表现得更加直接：当总 shots 数量固定时，扩大权重集合会压缩单个方向能够获得的测量次数，因此权重覆盖范围与单方向采样深度需要共同考虑。

此外，线性加权方法本身也存在一定局限。对于非凸的连续 Pareto 前沿，或者具有复杂离散结构的多目标组合优化问题，部分非支撑 Pareto 解难以通过单纯改变线性权重稳定地获得。因此，增加权重方向并不意味着能够完整恢复真实 Pareto 前沿。权重设计更适合作为组织搜索方向和改善目标空间覆盖的基础，而最终解集质量仍然受到求解器搜索能力、采样分布以及非支配解筛选过程等多方面因素的影响^[5-6,9]。

上述研究为本文的权重设计提供了基础。考虑到赛题具有五个优化目标，且量子采样总预算受到严格限制，本文工作的重点并不是单纯增加权重数量，而是在结构化覆盖与采样深度之间寻找合适的平衡。在后续方案中，规则离散的权重空间被用于建立第一阶段搜索基础，而不同方向的后续资源投入则结合实际采样结果进一步确定，从而将权重空间的先验结构与实例相关的采样反馈结合起来。

2.3 有限 shots 下的探索

在计算资源受到限制的优化问题中，如何平衡搜索范围与重点区域的深入搜索是一个普遍存在的问题。相关研究通常将其概括为探索与利用之间的平衡：探索强调扩大搜索范围，获取更多未知区域的信息；利用则强调根据已有信息，将更多计算资源投入当前表现较好的区域，以进一步提高获得高质量解的概率。不同问题中的具体实现方式有所差异，但其共同目标都是避免有限资源在过度分散和过度集中之间失衡。

这一问题在多目标优化中表现得更加复杂。为了获得具有良好覆盖性的 Pareto 前沿，算法通常需要从多个目标偏好方向展开搜索。增加搜索方向有助于改善目标空间的覆盖范围，但在总计算预算固定时，每个方向能够获得的计算资源也会随之减少。反之，如果仅选择少量方向进行深入搜索，虽然可以提高局部求解强度，却可能遗漏

其他具有价值的目标权衡区域。因此，多目标优化中的资源配置不仅需要考虑单个子问题的求解质量，还需要兼顾整个解集的多样性。

在基于 QAOA 的求解过程中，这一矛盾进一步体现为权重方向数量与单方向 shots 之间的竞争关系。对于给定权重和固定线路参数，量子线路的测量结果服从相应的概率分布。增加 shots 并不会改变这一分布本身，但能够增加对其进行观测的次数，使低概率候选解获得更多被测量到的机会。因而，单方向 shots 过少时，即使某个权重具有发现高质量候选解的潜力，也可能由于观测次数不足而无法充分表现。

已有资源分配方法通常可以分为静态分配和动态分配两类。静态方法在计算开始前确定各任务的预算，其中均匀分配具有实现简单和先验假设较少的优点，但难以利用不同搜索方向之间的实际差异。动态方法则根据运行过程中获得的信息逐步调整资源投入，使表现较好的区域获得更多计算机会。这类方法能够提高资源配置的针对性，但其效果依赖于前期信息是否足以反映不同搜索区域的真实价值。

对于多目标量子采样问题，直接采用均匀分配虽然能够保持各权重方向之间的预算公平，但并未考虑不同方向在实际采样结果上的差异。另一方面，如果在缺少充分信息时过早集中预算，也可能因为随机采样波动而错误地忽略部分潜在有效方向。因此，更合理的资源配置方式需要在前期保留一定搜索广度，通过初步采样获得方向信息，再根据反馈逐渐提高资源分配的针对性。

上述研究思路表明，在有限 shots 约束下，权重空间覆盖与单方向采样深度不能彼此独立地进行优化。较为可行的思路是将搜索过程划分为具有不同功能的阶段：前期侧重覆盖与信息获取，后期根据已有结果调整搜索重点。本文在此基础上进一步研究适用于多目标 QAOA 的分阶段采样机制，具体的阶段划分、权重筛选和预算配置方法将在第三章中介绍。

2.4 采样反馈驱动

动态资源分配的核心在于如何利用已有计算结果评价不同搜索区域的潜在价值。在经典优化、启发式搜索等相关研究中，自适应策略通常根据历史搜索结果不断更新对不同候选区域的评价，并据此调整后续计算资源。与完全静态的资源分配方式相比，这类方法能够利用运行过程中逐步获得的信息，使有限预算更多地投入具有潜力的搜索区域。

在多目标优化中，搜索区域的价值评价比单目标问题更加复杂。单目标优化通常可以直接根据目标函数值或改进幅度评价搜索效果，而多目标问题需要同时考虑收敛性和多样性。一个搜索方向可能产生大量候选解，但这些解在目标空间中高度集中；另一个方向产生的候选解数量较少，却可能补充当前 Pareto 前沿的稀疏区域。因此，仅依据样本数量、单一标量化能量或权重的几何位置，都难以完整评价一个方向对最终解集的实际价值。

相关多目标优化研究通常利用非支配关系、拥挤程度、参考方向关联关系或超体积贡献等信息评价候选解及搜索区域。其中，HV 贡献能够较直接地反映候选点对当前前沿覆盖范围的贡献，但其计算依赖当前全局解集，且在目标维数和候选规模增加后计算成本随之上升。拥挤距离等指标能够描述当前前沿的局部稀疏程度，但目标空间中的稀疏区域并不一定对应能够通过当前搜索机制有效发现的新解。对于基于多个权重方向的量子采样，还需要考虑反馈指标能否稳定地映射到具体权重，并用于后续 shots 分配^[7-8, 12]。

另一类较为直接的思路是根据各搜索方向产生非支配候选解的能力进行评价。与单纯比较最低能量或样本数量相比，非支配解数量能够在一定程度上反映一个方向产生多样目标权衡关系的能力；同时，该指标只依赖当前采样结果，不需要预先知道真实 Pareto 前沿，因此适合用于未知测试实例上的在线评价。不过，这类指标同样存在局限：方向内部的非支配解进入全局候选集合后仍可能被其他方向产生的解支配，因此局部非支配解数量并不能等同于该方向对最终 HV 的真实贡献。

由此可见，反馈指标在自适应采样中的作用更适合被理解为方向筛选和资源配置的参考依据，而不是对未来收益的精确预测。为了降低采样随机性和评价误差的影响，后续资源分配通常还需要在重点强化与方向多样性之间保留一定余量，避免将全部预算过早集中于极少数方向。

基于上述分析，本文进一步研究如何从第一阶段量子采样结果中提取方向级反馈，并将其用于后续权重筛选和 shots 分配。与依赖真实 Pareto 前沿或公开实例后验信息的方法不同，本文希望反馈机制能够直接作用于当前待求解实例，并在较低额外计算成本下完成方向评价。具体评价指标、方向保留规模及预算分配方法将在第三章给出，并在第四章通过实验分析其有效性。

2.5 大规模样本处理与 Pareto 前沿维护

在多目标优化算法中，求解过程通常会产生大量候选解，而最终输出只保留其中的非支配解。因此，除了优化算法本身的搜索能力外，候选解的目标值计算、重复样本处理、非支配关系判断以及 Pareto 前沿维护同样会影响整体求解效率。当候选样本规模较小时，这部分计算开销通常并不突出；随着样本数量和目标维数增加，经典后处理可能逐渐成为整个求解流程中的主要性能瓶颈^[5-6, 12-14]。

大规模候选集的处理通常包含两个计算密集环节。首先，需要将决策空间中的候选解映射到目标空间，即计算每个候选解对应的多个目标函数值。对于图结构组合优化问题，这一过程往往涉及大量节点状态读取和边关系计算。如果完全采用通用的逐边索引方式，在样本规模较大时会产生频繁的数据访问和中间数组构造。相关高性能计算方法通常通过批量计算、向量化处理、结构化数据布局以及针对特定拓扑的专用计算路径提高目标值计算效率。

另一个主要问题来自非支配筛选。最直接的方法需要对候选点之间的支配关系进行大量比较，其计算开销会随着候选集合规模快速增长。为降低这一成本，相关研究提出了多种改进思路，例如利用排序关系减少比较范围、采用分治方法处理不同候选子集、通过空间索引结构辅助支配查询，以及采用增量式方法维护动态 Pareto 前沿等。这些方法的共同思路是利用已有的顺序关系、空间关系或当前前沿信息，尽量避免对全部候选点进行重复的全局比较。

对于持续产生候选解的数据处理场景，增量式 Pareto 前沿维护具有较好的适用性。该类方法不必在每次加入新样本后重新对全部历史数据执行完整非支配筛选，而是维护当前已知的非支配集合，并判断新候选点与当前前沿之间的支配关系。当候选数据能够分批处理时，还可以先在局部数据块内部完成筛选，再将规模较小的局部前沿与已有全局前沿进行合并，从而减少参与后续高成本比较的候选点数量。

在增量式处理流程中，预筛选也是常见的优化思路。已有 Pareto 前沿包含了当前搜索过程积累的有效支配信息，可以用于提前删除一部分明显无效的候选点。不过，预筛选并非越复杂越好。更严格的过滤规则通常需要更多判断和中间计算，如果筛选本身的成本接近甚至超过后续非支配比较所节省的开销，整体运行时间反而可能增加。因此，在实际实现中，需要在过滤能力与额外计算成本之间进行权衡。

除算法复杂度外，实际运行效率还受到数据布局、内存访问和临时数组规模等因素影响。基于数组的批量计算虽然能够充分利用底层数值计算库，但如果频繁执行不连续索引或构造大规模临时布尔矩阵，也可能产生较高的数据搬运和内存分配成本。因此，对于大规模多目标样本处理，仅减少理论比较次数并不足以保证实际加速，还需要结合数据结构和底层计算方式综合考虑。

本赛题的 `main2` 同时具有大规模候选样本和规则 Ising 图结构两个特点。一方面，大量样本需要依次完成多目标值计算和非支配筛选，前一环节保留的无效候选点会继续增加后续 Pareto 判断的计算负担；另一方面，部分测试实例具有规则二维格点拓扑，为利用连续数据访问和批量运算改写通用能量计算过程提供了条件。

基于相关研究思路和赛题数据特点，本文主要从目标值计算和 Pareto 前沿维护两个层面进一步开展优化。在目标值计算阶段，利用规则格点拓扑减少通用边索引带来的额外开销；在前沿维护阶段，则结合分块处理、已有前沿辅助筛选、局部非支配筛选和增量式合并逐步压缩候选集合规模。上述方法的具体实现及其组合方式将在第三章中进一步介绍。

3 方案描述

3.1 总体方案

根据第二章的分析，本文将整体求解流程划分为 `main1` 和 `main2` 两部分。`main1` 面向 100000 shots 的预算约束，目标是在有限量子采样次数下获得高质量候选解；`main2` 面向大规模样本，负责高效完成目标值计算和 Pareto 前沿维护。两部分的优化对象不同，但都遵循同一思路：利用运行过程中已经获得的信息，减少无差别的平均投入。

`main1` 的流程为：构造 $H = 6$ 的 Simplex Lattice 权重集合，对全部方向执行第一阶段采样，根据方向内 ND 指标排序，选择 Top-K 进入第二阶段，并按指数衰减方式分配剩余 shots。`main2` 则采用规则格点快速能量计算与增量式前沿维护相结合的方式，依次完成分块处理、已有前沿预筛选、块内非支配筛选和双前沿交叉合并。

本章重点说明最终方案的实现方式。第二章已经给出各项设计的动机与问题分析，第四章则进一步讨论关键参数的选择过程以及替代方案的实验表现。

3.2 Simplex Lattice 权重设计

对于五目标优化问题，需要通过不同权重组合五个目标，从多个目标偏好方向构造标量化子问题。若直接随机生成权重，有限数量的方向可能在单纯形空间中分布不均，部分区域过于密集，而另一些区域缺少有效覆盖。因此，本文采用 Simplex Lattice 方法生成第一阶段使用的结构化权重集合。

Simplex Lattice 的基本思想是将权重空间按照固定粒度进行规则离散。对于五目标问题，每个权重向量包含 5 个分量，各分量非负且总和为 1。设离散参数为 H ，则每个权重分量只能取 0 、 $1/H$ 、 $2/H$ 、 $\dots$ 、 1 中的离散值，并从中枚举所有满足权重和为 1 的组合。当 $H = 6$ 时，单个权重分量的变化步长为 $1/6$ ，程序通过枚举 5 个非负整数分量，并保留分量之和等于 6 的组合，再统一除以 6，最终得到 210 个互不重复的权重方向。

例如，生成的权重集合中既包含仅强调单个目标的极端方向，也包含同时关注两个或多个目标的混合方向。前者能够搜索单一目标占主导时的极端解，后者则用于探索不同目标之间的折中区域。由于所有权重均由同一离散规则产生，不需要额外进行随机采样或人工筛选，因此能够形成较稳定的初始搜索空间。

在具体实现中，程序在量子采样开始前一次性生成完整的 210 个权重向量，并按照固定顺序保存。对于任意一个权重方向，分别使用其 5 个权重分量对 5 个 Ising 目标的局部场参数和耦合参数进行加权组合，得到当前方向对应的标量化 Ising 子问题。随后调用统一的 QAOA 采样流程，对该方向对应的问题哈密顿量进行采样。按照这一过程依次遍历全部 210 个方向，即可完成第一阶段的广覆盖搜索。

第一阶段对所有权重方向分配相同的 300 shots，一方面保证完整权重空间均得到初步探索，另一方面也为后续方向评价提供相对一致的比较基础。每个方向的采样结果均独立保存，并在后续处理中分别完成样本去重、目标值计算和方向内非支配筛选，由此得到用于第二阶段排序的 ND 指标。因此，Simplex Lattice 权重集合不仅承担第一阶段的搜索任务，也构成后续自适应资源分配的候选方向空间。

H 的取值决定权重空间的离散粒度，同时也影响权重数量与单方向采样深度。较小的 H 生成方向较少，可以为单个方向保留更多采样预算；增大 H 能够提高权重空间的离散分辨率，但也会增加方向数量。在本文的实验中， $H = 5$ 和 $H = 6$ 均表现出较好的搜索能力，相关对比过程将在第四章详细分析。综合权重覆盖范围和后续两阶

段采样需求，最终方案采用 $H = 6$ 生成的 210 个方向作为第一阶段搜索空间，并根据第一阶段反馈从中选择部分方向进入第二阶段，而不再对全部方向平均追加预算。

3.3 QAOA 子问题与采样实现

对于每个权重方向，程序先按照权重对五个目标的 Ising 系数进行线性组合，得到对应的加权子问题，再依据赛题提供的 QAOA 线路和迁移参数完成采样。本文保留官方工具中与线路构造、参数读取和基础缩放相关的通用实现，不再重复介绍其内部细节，而将优化重点放在权重组织与 shots 分配策略上^[15-17]。

同一阶段内，每个权重方向独立执行采样，并保存测量得到的比特串。第一阶段结果除用于形成候选解集外，还按方向保留，以便后续计算 ND 指标。第二阶段采样完成后，将两阶段样本统一汇总、去重并计算多目标值，再进入最终非支配筛选。

这样的实现保持了量子采样模块与资源分配模块之间的清晰边界。QAOA 线路本身不随方向筛选逻辑改变，算法的自适应性主要体现在权重选择和 shots 配置上，因而便于复现实验，也便于单独分析不同资源分配策略的影响。

3.4 两阶段自适应采样

第一阶段对 $H = 6$ 生成的 210 个方向分别分配 300 shots，共使用 63000 shots。采样完成后，对每个方向的测量结果计算其产生 ND 解的数量，作为指标进行第二轮采样方案分配。

程序按照 ND 指标从高到低对 210 个方向排序，选择前 74 个方向进入第二阶段。剩余 37000 shots 不再平均分配，而是按照方向排名采用指数衰减权重，分配到的采样数量随排名增大而递减，再通过归一化和整数化得到实际 shots，并对舍入误差进行修正，保证两阶段总采样量严格满足 100000 shots 的限制。

指数衰减分配保留了 Top-74 内部的排序信息：排名靠前的方向获得更多重复采样机会，中后部方向仍保留继续发现新解的预算。两阶段样本最终统一汇总并进入经典后处理流程，从而在同一候选集合上完成去重、目标值计算和非支配筛选。

Top-74 和指数衰减并非预先假定的唯一选择。第四章将结合补充方向、稀疏区域权重和其他方向组合实验说明最终参数的形成过程。

3.5 main2 后处理优化

main2 首先将大规模样本划分为若干数据块。每个数据块完成多目标值计算后，不直接与全部历史候选点合并，而是先利用已有前沿进行辅助预筛选，再执行块内非

支配筛选，仅保留局部前沿参与后续更新。这样能够控制单次高成本支配判断所面对的数据规模。

随着数据块不断处理，全局前沿也会逐步更新。新数据块进入完整局部筛选前，先利用当前前沿删除能够快速确认被支配的候选点。这里的预筛选保持轻量，其目标并非一次删除所有无效点，而是在较低额外开销下减少后续 ND 计算规模。

局部前沿形成后，与当前全局前沿执行交叉合并。由于两个输入集合内部已经分别非支配，合并时只需要检查集合之间的支配关系，无需重复比较各自内部的点。完成交叉判断后，删除被另一集合支配的元素，即可得到更新后的全局前沿。

对于规则二维格点实例，程序采用专门的快速能量计算路径。样本状态按照格点结构重排后，水平和垂直邻接关系通过连续切片获得，并结合批量布尔运算与矩阵计算完成多个目标的求值。程序在进入快速路径前检查图结构是否满足规则格点条件；若条件不满足，则回退到通用边索引实现。

上述模块共同构成最终 `main2` 流程。第四章不再重复实现细节，而是从优化路径和方法取舍角度说明分块、预筛选、交叉合并以及规则格点计算为何被保留。

4 结果与分析

4.1 实验设计

本章选取方案形成过程中最具代表性的实验进行分析。`main1` 重点回答三个与最终方案直接相关的问题：改变权重设计能否改善结果，两阶段采样能否缓解有限 shots 下覆盖范围与采样深度之间的矛盾，以及第二阶段应如何选择方向并分配剩余预算。`main2` 则考察不同处理流程对经典后处理效率的影响，并选择合适的方法应用于最终方案。

4.2 权重集合设计的影响

研究初期，本文首先对不同权重集合的搜索效果进行了比较。在 `demo` 的三轮采样方案下，直接使用官方权重列表前 100 个方向时，得分为 129.95；替换为随机生成的 100 个权重后，得分提升至 220.07。虽然这组结果来自方案探索的早期阶段，采样流程与最终方案并不完全相同，但两种权重集合之间的明显差距说明，在有限采样预算下，权重方向的选取会显著影响多目标 QAOA 对不同目标权衡区域的搜索能力。

因此，后续研究的重点由简单调整已有权重列表，逐渐转向构造覆盖更加稳定、结构更加明确的权重集合。

在此基础上，本文进一步研究 Simplex Lattice 方法生成的结构化权重。实验分别采用不同的离散参数 H 生成完整权重集合，其中 $H = 5$ 、 $H = 6$ 和 $H = 7$ 时分别得到 126、210 和 330 个权重方向。为了在相同采样预算下进行初步比较，首先从不同规模的完整权重集合中按比例选取 100 个方向，并为每个方向分配 1000 shots，实验结果如表 4-1 所示。

表 4-1 Simplex Lattice 方法按比例选取 100 个方向

权重方案	Score
$H = 5$	224.424303
$H = 6$	224.263312
$H = 7$	221.148639

从实验结果可以看出，三组 Simplex Lattice 权重均取得了较好的结果。其中， $H = 5$ 时按比例选取 100 个方向的效果最好，但这一现象需要结合完整权重集合的规模进行分析。 $H = 5$ 仅包含 126 个方向，选取其中 100 个后仍保留了大部分原始方向；随着 H 增大，完整权重集合的规模迅速增加，在固定选取 100 个方向的情况下，实际保留的方向比例随之下降。因此，这一组实验不仅反映了离散参数 H 的影响，也受到方向截取比例的影响，不能据此简单判断较小的 H 一定具有更好的搜索能力。

随后，本文比较了“选取部分方向并增加单方向 shots”与“保留完整方向集合”两种策略。对于 $H = 5$ ，采用 100×1000 的采样方案时得分为 224.95，而完整使用 126 个方向并采用 126×793 的近似等预算方案时，得分提升至 226.31。 $H = 6$ 的实验也表现出相同趋势，即在相近总预算下，保留完整权重集合的效果优于从中截取部分方向后增加单方向采样次数。

这一结果表明，在当前问题中，部分权重方向的缺失难以完全通过增加已有方向的 shots 加以补偿。对单个方向增加采样次数能够提高其测量分布中低概率候选解被观察到的机会，但无法替代其他权重方向所对应的目标偏好。对于多目标优化任务而言，方向覆盖本身具有独立价值，因此后续方案优先保留完整的结构化权重集合，再进一步考虑如何对不同方向实施差异化的资源分配。

综合不同 H 的实验结果， $H = 5$ 与 $H = 6$ 体现出不同的资源配置特点。 $H = 5$ 仅包含 126 个方向，在固定总预算下能够为单个方向保留更大的采样深度； $H = 6$ 则

生成 210 个方向，具有更细的权重空间离散粒度和更充分的方向覆盖。继续增大到 $H = 7$ 后，方向数量达到 330 个，在 100000 shots 的总预算限制下，单方向能够获得的采样次数进一步下降，搜索覆盖与采样深度之间的矛盾更加突出。

基于上述实验，本文最终选择 $H = 6$ 生成的 210 个完整权重方向作为后续方案的初始搜索空间。这一选择并不意味着对全部方向持续平均投入预算，而是希望首先利用较完整的结构化权重集合建立搜索覆盖基础，再通过第一阶段实际采样结果识别不同方向的搜索价值，并在第二阶段重新配置剩余资源。由此，权重集合设计问题进一步转化为有限 shots 下的阶段划分与方向筛选问题，后续实验将在此基础上展开。

4.3 两阶段采样与 Top-74 选择

若将 100000 shots 平均分配给 210 个方向，每个方向只能获得约 476 次采样。这种方式保留了完整覆盖，却隐含了所有方向具有相同继续采样价值的假设。第一阶段结果显示，不同方向产生非支配候选解的能力存在明显差异，因此继续平均投入并不合理，这里本文考虑采用两轮采样。

最终方案先对 210 个方向分别采样 300 shots，共使用 63000 shots。第一阶段既保留完整权重覆盖，也为方向评价提供基础样本。随后按照方向内 ND 指标排序，从 210 个方向中筛选表现较好的方向进入第二阶段。这样，第二阶段的选择依据来自当前实例自身的量子采样结果，而不是预先固定的经验权重。

方向数量 K 决定第二阶段的搜索宽度。 K 过小，剩余预算虽然更加集中，但搜索范围容易过快收缩； K 过大，又会接近重新平均分配预算，削弱第二阶段的意义。经过多轮实验， $K = 74$ 在方向多样性与强化深度之间表现出较好的平衡，因此最终采用 Top-74 作为第二阶段方向集合，并且本文在两轮中采用不同的 QAOA 层数，第一轮选取层数为 5，第二轮选取层数为 4，在时间与结果之间取得很好的平衡，相关结果整理在表 4-2 中。

表 4-2 调整第二轮采样权重以及 QAOA 层数结果

权重	采样数	QAOA层数	Score
[210,37]	[300,1000]	[5,5]	228.29057
[210,37]	[300,1000]	[5,4]	229.31326
[210,50]	[300,740]	[5,5]	228.35131
[210,50]	[300,740]	[5,4]	229.25455
[210,74]	[300,500]	[5,5]	228.09295
[210,74]	[300,500]	[5,4]	229.44517

4.4 补充方向与指数衰减实验

确定 Top-K 框架后，本文曾尝试在高 ND 方向之外加入少量补充方向，希望第二阶段在集中强化的同时保留额外探索能力。代表性实验中，Top-60 加 14 个补充方向得到 228.282474，Top-68 加 6 个补充方向得到 228.426283，均低于最终方案。

这些结果说明，第二阶段方向数量不能只从多样性角度理解。第一阶段已经覆盖全部 210 个方向，因此第二阶段新增方向并非没有代价的额外探索，而会直接减少高价值方向能够获得的 shots。只有当补充方向发现新 Pareto 解的收益足以抵消重点方向采样深度下降带来的损失时，这种扩展才可能有效；现有实验并未出现这种情况。

本文还尝试根据当前前沿的稀疏区域构造补充权重，但同样没有得到稳定提升。目标空间中的稀疏区域并不意味着附近一定存在容易通过线性标量化发现的新解，尤其在离散多目标问题中，部分缺失点可能属于非支撑 Pareto 解。继续增加权重方向无法保证这些区域被有效访问，却会带来确定的预算竞争。

基于上述实验，最终方案放弃额外方向补充，保留统一的 Top-74 排名筛选。第二阶段进一步按照第一阶段排名，以指数衰减方式分配剩余 37000 shots：排名靠前的方向获得更多采样机会，靠后的方向仍保留一定预算。该方案最终取得 229.624789 的得分。

表 4-3 第二轮采样分配方案实验结果

权重	采样数	QAOA层数	分配方式	Score
[210,74]	[300,500]	[5,4]	均匀	229.44517
[210,74]	[300,500]	[5,4]	线性衰减	229.60040
[210,74]	[300,500]	[5,4]	指数衰减	229.62479

回顾整个实验过程，性能提升并非来自某个孤立参数，而是来自资源配置方式的逐步调整：首先通过 Simplex Lattice 方法生成权重，更好的覆盖不同方向，第一阶段采样随后提供方向价值信息，Top-74 据此收缩重点搜索范围，指数衰减则进一步区分入选方向的采样强度。图 4-1 概括了这一过程。



图 4-1 main1 方法路径

4.5 main2 优化过程

main2 的优化始于对 baseline 实际瓶颈的分析。测试表明，主要时间开销集中在多目标 Ising 能量计算和非支配筛选两个环节，因此后续工作分别围绕数据访问方式与候选集合规模展开。

首先将一次性全局处理改为分块流式处理。每个数据块完成目标值计算后先提取局部前沿，再与当前全局前沿合并。这样既降低了单次处理规模，也使已有全局前沿能够参与后续数据块的过滤。在此基础上加入轻量预筛选，使明显被当前前沿支配的候选点不再进入完整的块内 ND 判断。

局部前沿与全局前沿合并时，两个集合内部已经分别完成非支配筛选。若将二者简单拼接后重新执行完整筛选，会重复判断大量已知关系。因此，本文采用双前沿交叉合并，只处理两个集合之间的支配关系。该方法与分块处理相互配合，使 Pareto 前沿能够以增量方式维护。

能量计算方面，赛题大规模实例具有规则二维格点结构。本文据此增加快速路径，将水平和垂直邻接关系转换为连续数组切片，并结合批量运算完成多个目标的能量计算。相比通用边索引方式，这一实现减少了临时数组和不连续访存。程序仍保留结构检查与通用回退路径，避免优化依赖未经验证的结构假设。

优化过程中还测试了 full prefilter、增量 ND、复杂 early-bound、随机数预分配以及部分 Numba 实现，但这些方法并未全部带来稳定收益。实际运行时间同时受到判断开销、数组规模、内存访问和 NumPy 批量计算效率影响，局部操作次数减少并不

必然转化为整体加速。最终方案保留了额外开销较低、流程清晰且能够稳定减少无效计算的组合。图 4-2 展示了 main2 的主要优化路径。



图 4-2 main2 优化路径

4.6 本章小结

本章通过代表性实验梳理了最终方案的形成过程。main1 的研究重点由早期权重选择逐步转向结构化覆盖和反馈驱动的预算分配。高预算参考实验表明， $H = 6$ 的 210 个方向具有较高搜索潜力；在正式预算约束下，两阶段采样、Top-74 筛选和指数衰减分配进一步解决了有限 shots 的配置问题。main2 则从完整处理链路出发，通过规则格点快速计算、分块处理、轻量预筛选和双前沿交叉合并减少无效开销。两部分实验共同说明，最终性能来自对搜索空间、采样反馈和中间计算状态的持续利用。

5 创新点总结

5.1 结构化权重空间设计

第一项创新是将权重选择从经验性的列表截取转化为规则的单纯形离散，并进一步与后续反馈筛选结合。Simplex Lattice 提供稳定且可解释的初始覆盖，降低有限方向在局部区域过度集中的风险；第一阶段采样则在统一的权重基础上比较不同方向的实际表现。

这一设计的价值并不只是增加权重数量，而是将覆盖与强化的功能明确分开。结构化权重负责降低先验遗漏风险，后续资源分配则把有限 shots 集中到当前实例中更有潜力的方向，使几何先验能够自然过渡到实例反馈。

5.2 采样反馈驱动的两阶段量子采样

第二项创新是把固定预算下的量子采样组织为探索与强化两个阶段。第一阶段同时承担候选解生成和方向信息收集功能，第二阶段利用已有反馈调整资源投入。由此，第一阶段样本既是最终候选集合的一部分，也是后续决策的数据来源。

与固定权重、固定 shots 的单轮采样相比，该机制允许不同实例形成不同的第二阶段重点方向。资源配置由采样前的静态决策转变为采样过程中的动态调整，从而增强了方法对隐藏实例的适应能力。

5.3 非支配解发现能力与分层预算调度

第三项创新是以方向内非支配解发现能力作为反馈信号，并将该信号连续用于方向筛选和预算分配。ND 指标无需真实前沿，可直接从当前实例的采样结果中获得；Top-74 先完成方向集合层面的筛选，指数衰减再进一步区分入选方向的采样强度。

main1 的关键并非某个单独参数，而是一条完整的信息传递链：结构化权重提供初始搜索空间，第一阶段采样产生反馈，ND 指标将反馈转化为排序，Top-74 和指数衰减再把排序结果转化为第二阶段资源配置。

5.4 经典后处理协同优化

main2 的创新体现在对完整经典处理链路的协同调整。规则格点快速计算利用问题拓扑降低目标求值成本，分块处理改变候选数据的组织方式，轻量预筛选和局部 ND 逐层压缩集合规模，双前沿交叉合并则利用中间集合已知的非支配性质避免重复比较。

这些方法并非简单叠加。前一环节减少的数据规模会直接降低后一环节成本，而交叉合并又建立在局部筛选已经完成的基础上。因此，最终方案体现的是数据结构、筛选顺序和前沿维护方式之间的协同优化。

5.5 全文总结与创新价值

本文围绕有限量子采样预算下的解集质量和大规模经典后处理效率两个问题展开研究。main1 通过结构化权重覆盖、采样反馈评价和分层预算分配提高 shots 利用效率；main2 则利用规则格点结构和增量式前沿维护减少目标计算与支配判断中的无效开销，最终方案在本地实验中取得了 229.624789 的 main1 得分，并在初赛正式评测中取得 210.59139 的最终成绩。

6 参考文献

- [1] CEREZO M, ARRASMITH A, BABBUSH R, et al. Variational quantum algorithms[J]. Nature Reviews Physics, 2021, 3: 625-644.
- [2] FARHI E, GOLDSTONE J, GUTMANN S. A quantum approximate optimization algorithm [EB/OL]. 2014. <https://arxiv.org/abs/1411.4028>.
- [3] ZHOU L, WANG S T, CHOI S, et al. Quantum approximate optimization algorithm: performance, mechanism, and implementation on near-term devices[J]. Physical Review X, 2020, 10(2): 021067.
- [4] XIAO J, SUKULTHANASORN N, NOMURA R, et al. An efficient quantum approximate optimization algorithm with fixed linear ramp schedule for truss structure optimization [EB/OL]. 2025. <https://arxiv.org/abs/2502.16769>.
- [5] DEB K. Multi-objective optimization using evolutionary algorithms: an introduction [M]. Springer, 2011: 3-34.
- [6] EMMERICH M T M, DEUTZ A H. A tutorial on multiobjective optimization: fundamentals and evolutionary methods[J]. Natural Computing, 2018, 17(3): 585-609.
- [7] ZITZLER E, THIELE L. Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach[J]. IEEE Transactions on Evolutionary Computation, 1999, 3(4): 257-271.
- [8] AUGER A, BADER J, BROCKHOFF D, et al. Theory of the hypervolume indicator: optimal μ -distributions and the choice of the reference point[C]//Foundations of Genetic Algorithms (FOGA 2009). New York: ACM, 2009: 87-102.
- [9] ZHANG Q, LI H. MOEA/D: a multiobjective evolutionary algorithm based on decomposition[J]. IEEE Transactions on Evolutionary Computation, 2007, 11(6): 712-731.
- [10] DAS I, DENNIS J E. Normal-boundary intersection: a new method for generating the Pareto surface in nonlinear multicriteria optimization problems[J]. SIAM Journal on Optimization, 1998, 8(3): 631-657.
- [11] KOTIL A, PELOFSKE E, RIEDMÜLLER S, et al. Quantum approximate multi-objective optimization[J]. Nature Computational Science, 2025, 5(12): 1168-1177.
- [12] DEB K, PRATAP A, AGARWAL S, et al. A fast and elitist multiobjective genetic algorithm: NSGA-II[J]. IEEE Transactions on Evolutionary Computation, 2002, 6(2): 182-197.
- [13] ZITZLER E, LAUMANN S, THIELE L. SPEA2: improving the strength Pareto evolutionary

- algorithm[R]. Zurich: ETH Zurich, 2001.
- [14] KNOWLES J D, CORNE D W. Approximating the nondominated front using the Pareto archived evolution strategy[J]. Evolutionary Computation, 2000, 8(2): 149-172.
- [15] AKSHAY V, RABINOVICH D, CAMPOS E, et al. Parameter concentrations in quantum approximate optimization[J]. Physical Review A, 2021, 104(1): L010401.
- [16] GALDA A, LIU X, LYKOV D, et al. Transferability of optimal QAOA parameters between random graphs[C]//2021 IEEE International Conference on Quantum Computing and Engineering (QCE). Piscataway: IEEE, 2021: 171-180.
- [17] SURESHBABU S H, HERMAN D, SHAYDULIN R, et al. Parameter setting in quantum approximate optimization of weighted problems[J]. Quantum, 2024, 8: 1231.

附录A main1 主要功能代码

本附录选取与论文中 main1 部分核心方案直接对应的关键实现代码。完整程序随作品代码一并提交。

A.1. 生成 Simplex Lattice 权重集合

```
def make_simplex_lattice_bank(
    k: int = 5,
    H: int = SIMPLEX_H,
    total: int = FIRST_ROUND_WEIGHTS,
    seed: int = 226,
    eps: float = 1e-12,
) -> np.ndarray:
    """生成 Simplex Lattice 权重集合。

    对 k 目标问题，枚举所有非负整数向量 a，使得 sum(a)=H，
    并令 lambda=a/H。若 total 小于完整 lattice 大小，则按支撑集
    大小分层抽样；在本方案中 H=6、k=5、total=210，正好使用完整集合。
    """

    if H <= 0:
        raise ValueError("H must be positive")
    if total <= 0:
        raise ValueError("total must be positive")

    rng = np.random.default_rng(seed)
    compositions: List[List[int]] = []

    def rec(remaining: int, dim: int, prefix: List[int]) -> None:
        if dim == k - 1:
            compositions.append(prefix + [remaining])
            return
        for value in range(remaining + 1):
            rec(remaining - value, dim + 1, prefix + [value])
```

```

rec(H, 0, [])
lattice = np.asarray(compositions, dtype=np.float64) / float(H)
lattice_size = int(lattice.shape[0])
if total > lattice_size:
    raise ValueError(f"total={total} exceeds lattice size={lattice_size}")

support_size = np.count_nonzero(lattice > 0.0, axis=1)
groups: Dict[int, np.ndarray] = {
    s: np.where(support_size == s)[0] for s in range(1, k + 1)
}

raw_counts = {s: len(idx) for s, idx in groups.items() if len(idx) > 0}
ideal = {s: total * raw_counts[s] / lattice_size for s in raw_counts}
take = {s: int(np.floor(ideal[s])) for s in raw_counts}

remaining = total - sum(take.values())
frac_order = sorted(raw_counts, key=lambda s: ideal[s] - take[s], reverse=True)
for s in frac_order:
    if remaining <= 0:
        break
    if take[s] < raw_counts[s]:
        take[s] += 1
        remaining -= 1

while remaining > 0:
    candidates = [s for s in raw_counts if take[s] < raw_counts[s]]
    if not candidates:
        raise RuntimeError("Failed to allocate lattice subset")
    s = max(candidates, key=lambda x: raw_counts[x] - take[x])
    take[s] += 1
    remaining -= 1

chosen_indices: List[int] = []
for s in range(1, k + 1):
    n_take = take.get(s, 0)
    if n_take > 0:
        chosen = rng.choice(groups[s], size=n_take, replace=False)
        chosen_indices.extend(chosen.tolist())

rng.shuffle(chosen_indices)
lambdas = lattice[chosen_indices]

# 极小 eps 用于避免个别后续实现对严格零权重处理不稳。
lambdas = np.maximum(lambdas, eps)
lambdas = lambdas / lambdas.sum(axis=1, keepdims=True)
if lambdas.shape != (total, k):
    raise AssertionError(f"lambdas shape={lambdas.shape}, expected={{total, k}}")
return np.asarray(lambdas, dtype=np.float64)

```

A.2. 计算权重方向非支配解数量

```

def count_direction_nd(
    unique_spin_blocks: List[np.ndarray],
    edges: np.ndarray,
    weights: np.ndarray,
    h: np.ndarray,
    lower: np.ndarray,

```

```

        upper: np.ndarray,
    ) -> np.ndarray:
        """计算每个权重方向在第一轮采样中产生的方向内 ND 解数量。"""
        nd_counts: List[int] = []
        for unique_spins in unique_spin_blocks:
            energies = energy_batch_fast(unique_spins, edges, weights, h)
            objectives = normalize_energies(energies, lower, upper)
            nd_idx = pg_non_dominated_indices(objectives)
            nd_counts.append(int(nd_idx.size))
        return np.asarray(nd_counts, dtype=np.int64)

```

A.3. 指数衰减分配第二轮 shots

```

def allocate_shots_exp(
    num_weights: int,
    total_shots: int,
    min_shots: int,
    max_shots: int,
    decay: float = 4.0,
) -> List[int]:
    """按排名指数衰减分配第二轮 shots。

    排名越靠前的权重获得越多 shots; 所有权重至少获得 min_shots,
    至多获得 max_shots。函数会修正取整误差, 确保总 shots 精确等于 total_shots。
    """

    n = int(num_weights)
    total = int(total_shots)
    smin = int(min_shots)
    smax = int(max_shots)

    if n <= 0:
        return []
    if total < n * smin:
        raise ValueError(f"total_shots={total} < n*min_shots={n * smin}")
    if total > n * smax:
        raise ValueError(f"total_shots={total} > n*max_shots={n * smax}")

    shots = np.full(n, smin, dtype=np.int64)
    remaining = total - n * smin
    if remaining == 0:
        return shots.tolist()

    rank = np.arange(n, dtype=np.float64)
    weights = np.exp(-float(decay) * rank / max(n - 1, 1))
    weights /= weights.sum()
    shots += np.floor(remaining * weights).astype(np.int64)
    shots = np.minimum(shots, smax)

    deficit = total - int(shots.sum())
    while deficit > 0:
        updated = False
        for i in range(n):
            if shots[i] < smax:
                shots[i] += 1
                deficit -= 1
                updated = True
                if deficit == 0:

```

```

        break
    if not updated:
        break

    return np.sort(shots)[::-1].tolist()

```

附录B main2 主要功能代码

本附录选取与论文中 main2 部分核心方案直接对应的关键实现代码。完整程序随作品代码一并提交。

B.1. 基于已有前沿的轻量预筛选

```

def _filter_not_dominated_by_pool(objs: np.ndarray, pool: np.ndarray) -> np.ndarray:
    """删除已被当前全局前沿支配的候选目标向量。"""
    if pool is None or pool.size == 0 or objs.size == 0:
        return objs
    keep = np.ones((objs.shape[0],), dtype=bool)
    for point in pool:
        idx = np.nonzero(keep)[0]
        if idx.size == 0:
            break
        dominated = np.all(objs[idx] >= point, axis=1)
        if np.any(dominated):
            keep[idx[dominated]] = False
    return objs[keep]

```

B.2. 双前沿交叉合并

```

def _merge_two_nd_pools_cross(pool: np.ndarray, local: np.ndarray) -> np.ndarray:
    """合并两个各自已经非支配的前沿集合，只检查集合之间的交叉支配关系。"""
    if pool is None or pool.size == 0:
        return np.asarray(local, dtype=np.float64)
    if local is None or local.size == 0:
        return np.asarray(pool, dtype=np.float64)

    pool = np.ascontiguousarray(pool, dtype=np.float64)
    local = np.ascontiguousarray(local, dtype=np.float64)
    n_pool = int(pool.shape[0])
    n_local = int(local.shape[0])

    if n_pool * n_local <= 8192:
        keep_local = np.ones((n_local,), dtype=bool)
        for point in pool:
            idx = np.nonzero(keep_local)[0]
            if idx.size == 0:
                break
            dominated = np.all(local[idx] >= point, axis=1)
            if np.any(dominated):
                keep_local[idx[dominated]] = False
        local = local[keep_local]
    if local.size == 0:
        return pool

```



```

keep_pool = np.ones((n_pool,), dtype=bool)
for point in local:
    idx = np.nonzero(keep_pool)[0]
    if idx.size == 0:
        break
    dominated = np.all(pool[idx] >= point, axis=1)
    if np.any(dominated):
        keep_pool[idx[dominated]] = False
return np.vstack([pool[keep_pool], local])

block = 384
keep_local = np.ones((n_local,), dtype=bool)
for start in range(0, n_local, block):
    end = min(start + block, n_local)
    blk = local[start:end]
    dominated = np.any(np.all(blk[:, None, :] >= pool[None, :, :], axis=2), axis=1)
    keep_local[start:end] = ~dominated
local = local[keep_local]
if local.size == 0:
    return pool

n_pool = int(pool.shape[0])
keep_pool = np.ones((n_pool,), dtype=bool)
for start in range(0, n_pool, block):
    end = min(start + block, n_pool)
    blk = pool[start:end]
    dominated = np.any(np.all(blk[:, None, :] >= local[None, :, :], axis=2), axis=1)
    keep_pool[start:end] = ~dominated

return np.vstack([pool[keep_pool], local])

```

B.3. 随机样本前沿计算与 HV 评估

```

def _large_random_frontier_hv_bool_opt(
    problem: IsingMOOProblem,
    *,
    shots: int = 200000,
    chunk_size: int = 4096,
    rng_seed: int = 2026,
    ref: float = HV_REF,
) -> Dict[str, object]:
    """main2 的随机样本前沿计算与 HV 评估。

```

该实现首先尝试识别规则二维格点结构，并在满足条件时使用切片方式批量计算水平边和垂直边贡献；随后对候选目标向量进行分块预筛选、局部 ND 筛选和全局前沿交叉合并。

```

    rng = np.random.default_rng(int(rng_seed))
    lower_bounds, upper_bounds = objective_extrema(problem)
    lo = np.asarray(lower_bounds, dtype=np.float64)
    hi = np.asarray(upper_bounds, dtype=np.float64)
    span = np.maximum(hi - lo, 1e-12)

    weights_scaled = np.ascontiguousarray(problem.weights / span[:, None], dtype=np.float64)
    h_scaled = np.ascontiguousarray(problem.h / span[:, None], dtype=np.float64)
    const = (-(np.sum(problem.weights, axis=1) + np.sum(problem.h, axis=1)) - lo) / span

```

```

field_matrix = np.ascontiguousarray(h_scaled.T, dtype=np.float64)

edges = np.asarray(problem.edges, dtype=np.int64)
u = np.ascontiguousarray(edges[:, 0])
v = np.ascontiguousarray(edges[:, 1])
k = int(problem.k)
n = int(problem.n)
a = int(problem.a)
bdim = int(problem.b)

delta = v - u
horizontal_mask = delta == 1
vertical_mask = delta == bdim
use_grid_fast = (
    a * bdim == n
    and int(np.count_nonzero(horizontal_mask)) == a * (bdim - 1)
    and int(np.count_nonzero(vertical_mask)) == (a - 1) * bdim
    and int(np.count_nonzero(horizontal_mask) + np.count_nonzero(vertical_mask))
    == int(edges.shape[0])
)
# 显式初始化分支专用矩阵，避免静态检查器报告“赋值前引用”。
weight_horizontal: np.ndarray | None = None
weight_vertical: np.ndarray | None = None
edge_weight_matrix: np.ndarray | None = None

if use_grid_fast:
    weight_horizontal = np.ascontiguousarray(weights_scaled[:, horizontal_mask].T)
    weight_vertical = np.ascontiguousarray(weights_scaled[:, vertical_mask].T)
else:
    edge_weight_matrix = np.ascontiguousarray(weights_scaled.T)

remaining = int(shots)
nd_pool = np.zeros((0, k), dtype=np.float64)
n_points = 0
t0 = time.perf_counter()

while remaining > 0:
    batch_size = min(int(chunk_size), remaining)
    bits = rng.random((batch_size, n)) < 0.5

    objs = bits @ field_matrix
    if use_grid_fast:
        assert weight_horizontal is not None and weight_vertical is not None
        grid = bits.reshape(batch_size, a, bdim)
        eq_h = (grid[:, :, :-1] == grid[:, :, 1:]).reshape(batch_size, -1)
        eq_v = (grid[:, :-1, :] == grid[:, 1:, :]).reshape(batch_size, -1)
        objs += eq_h @ weight_horizontal
        objs += eq_v @ weight_vertical
    else:
        assert edge_weight_matrix is not None
        edge_equal = bits[:, u] == bits[:, v]
        objs += edge_equal @ edge_weight_matrix

    objs *= 2.0
    objs += const[None, :]
    objs = np.asarray(objs, dtype=np.float64)

```

```

if nd_pool.size:
    prefilter_pool = nd_pool
    if prefilter_pool.shape[0] > 128:
        idx = np.argpartition(np.sum(prefilter_pool, axis=1), 128)[:128]
        prefilter_pool = prefilter_pool[idx]
    objs = _filter_not_dominated_by_pool(objs, prefilter_pool)

if objs.size:
    local_front = objs[pg_non_dominated_indices(objs)]
    nd_pool = _merge_two_nd_pools_cross(nd_pool, local_front)

n_points += batch_size
remaining -= batch_size

elapsed = time.perf_counter() - t0
nd_pool = np.asarray(lexsort_rows(nd_pool), dtype=np.float64)
hv = float(hypervolume_pygmo(nd_pool, ref=ref))
return {
    "shots": int(shots),
    "chunk_size": int(chunk_size),
    "n_points": int(n_points),
    "nd_count": int(nd_pool.shape[0]),
    "hv": hv,
    "frontier_objectives_norm": nd_pool.tolist(),
    "elapsed_s": float(elapsed),
}

```